REPORTE FINAL

Reporte de trabajo

Pilas y colas en Java · Estructura de Datos

ESTUDIANTE	ID	PUNTAJE
david vargas criollo	1092953	127 / 134

NOTA (0 A 5)	EJERCICIOS COMPLETOS	RESPUESTAS VISTAS
4.7	134 / 134	7

INICIO	GENERADO
3/11/2026, 6:25 a. m.	3/11/2026, 9:38 p. m.

Módulo	Puntos	Completos	Intentos
Dos reglas para salir	5 / 5	5 / 5	6
La pila	3 / 4	4 / 4	6
Pila y cola con arreglo	16 / 17	17 / 17	44
ArrayList como pila y cola	8 / 9	9 / 9	11
Lista enlazada como pila y cola	7 / 7	7 / 7	9
¿Qué tan rápido es? La notación O	5 / 5	5 / 5	6
Interfaz vs. clase	10 / 10	10 / 10	20
El mapa de las colecciones	9 / 9	9 / 9	13
La clase Stack	4 / 5	5 / 5	8
Excepciones y try-catch	4 / 4	4 / 4	5
La cola y la interfaz Queue	6 / 7	7 / 7	11
Capacidad y LinkedList	4 / 4	4 / 4	

Powered by Netlify

Deque y ArrayDeque	7 / 8	8 / 8	16
¿Toda cola es FIFO?	3 / 3	3 / 3	3
Colas de prioridad	11 / 11	11 / 11	13
Repaso: ¿verdadero o falso?	16 / 16	16 / 16	27
Reto: usar pilas y colas de verdad	5 / 6	6 / 6	8
¿Cuál usarías?	4 / 4	4 / 4	7

Reflexión sobre su proyecto

(sin respuesta)

Detalle de respuestas

Dos reglas para salir

Opción múltiple · **Correcto** · 1 intento(s)

Escribes "hola", luego "mundo", luego "!" y oprimes Ctrl+Z. ¿Qué se deshace primero?

```
"!", porque fue lo último
```

Opción múltiple · **Correcto** · 1 intento(s)

En la fila de una taquilla, ¿qué regla se cumple?

```
FIFO
```

Opción múltiple · **Correcto** · 1 intento(s)

¿Qué significa que pila y cola sean un TAD?

```
Que definen una regla de comportamiento, sin decir cómo se guardan
```

Opción múltiple · **Correcto** · 2 intento(s)

Una pila hecha con arreglo y otra hecha con nodos reciben push(10), push(20), push(30) y luego pop(). ¿Qué devuelve cada una?

```
30 las dos
```

Opción múltiple · **Correcto** · 1 intento(s)

¿Qué define un tipo abstracto de datos (TAD)?

```
Las operaciones y la regla, sin decir cómo se guardan los datos
```

La pila

Predice la salida · **Respuesta vista** · 3 intento(s)

¿Qué imprime este código? Escribe una línea por cada línea de la consola.

```
pop: esta quitando y devolviendo a la cima. quitar el de arriba (30).

peek: esta mostrando la cima sin quitarla (30).

size: esta mostrando cuantos hay (30).
```

Opción múltiple · **Correcto** · 1 intento(s)

¿Qué es underflow?

```
Intentar pop o peek en una pila vacía
```

Opción múltiple · **Correcto** · 1 intento(s)

Haces peek() tres veces seguidas. ¿Qué le pasa a la pila?

```
No cambia
```

Opción múltiple · **Correcto** · 1 intento(s)

¿En qué pila puede ocurrir overflow?

```
En una de tamaño fijo, hecha con arreglo
```

Pila y cola con arreglo

Opción múltiple · **Correcto** · 1 intento(s)

En PilaArreglo, ¿qué valor tiene tope cuando la pila está vacía?

```
-1
```

Predice la salida · **Correcto** · 2 intento(s)

¿Qué imprime? Son 3 líneas.

```
3
9
8
```

Escribe código · **Correcto** · 1 intento(s)

Escribe las dos instrucciones que van dentro de push, después del if, para guardar x en la cima.

```
tope++;
datos [tope]=x;
```

Opción múltiple · **Correcto** · 1 intento(s)

En ColaArreglo, ¿para qué sirve % datos.length?

```
Para que frente y fin vuelvan a la casilla 0 al llegar al final
```

Predice la salida · **Respuesta vista** · 19 intento(s)

¿Qué imprime? Son 3 líneas. Fíjate en cuántas casillas hay.

```
10
20
```

En NetBeans · **Correcto** · 4 intento(s)

Tu propia pila. En NetBeans crea la clase PilaArreglo tal como está arriba, escribiéndola tú. Luego, en tu clase con main, escribe este código, ejecuta y anota aquí toda la salida.

```
30
20
overflow: la pila esta llena
```

Opción múltiple · **Correcto** · 1 intento(s)

¿Por qué tope se declara private?

```
Para que solo push y pop lo muevan y la regla no se rompa
```

Opción múltiple · **Correcto** · 1 intento(s)

En pop(), ¿qué hace return x;?

```
Entrega x a quien llamó el método y termina el método
```

Predice la salida · **Correcto** · 1 intento(s)

¿Qué imprime este recorrido equivocado? Son 4 líneas.

```
10
20
30
0
```

Predice la salida · **Correcto** · 1 intento(s)

Con el mismo arreglo del ejercicio anterior, ¿qué imprime el recorrido correcto?

```
20
10
```

Escribe código · **Correcto** · 3 intento(s)

Escribe el ciclo for que muestra la pila de la cima al fondo, usando tope y el arreglo datos.

```
for(int i = tope; i ⩾ 0; i--) { System.out.println(datos[i]);}
```

Opción múltiple · **Correcto** · 1 intento(s)

int[] datos = new int[4]; Después de 3 push, ¿cuánto vale datos.length?

```
4
```

Opción múltiple · **Correcto** · 1 intento(s)

¿Cómo se escribe el tamaño de un arreglo llamado datos?

```
datos.length
```

Predice la salida · **Correcto** · 1 intento(s)

¿Qué imprime? Son 4 líneas.

```
1
0
2
0
```

Opción múltiple · **Correcto** · 3 intento(s)

Una cola con un arreglo de 5 casillas tiene fin = 3. Haces offer. ¿Cuánto vale fin ahora?

```
4
```

Escribe código · **Correcto** · 1 intento(s)

Escribe el cuerpo del método peek(): si la pila está vacía, avisa y devuelve -1; si no, devuelve la cima sin sacarla.

```
if (tope == -1) { System.out.println("la pila esta vacia");
return -1;
}
return datos[tope];
```

En NetBeans · **Correcto** · 2 intento(s)

Completa tu PilaArreglo. Agrégale en NetBeans los métodos peek() y size() de esta página. Luego ejecuta este main y anota toda la salida.

```
la pila esta vacia
-1
9
9
2
```

ArrayList como pila y cola

Opción múltiple · **Correcto** · 1 intento(s)

Un ArrayList con capacidad 4 ya tiene 4 datos. Haces add. ¿Qué pasa?

```
Crea un arreglo más grande, copia los datos y guarda el nuevo
```

Opción múltiple · **Correcto** · 1 intento(s)

¿Por qué un ArrayList es lento cuando se usa como cola?

```
Porque remove(0) obliga a correr a todos los demás un puesto
```

Predice la salida · **Respuesta vista** · 3 intento(s)

¿Qué imprime? Son 2 líneas.

```
10, 20
20
```

Escribe código · **Correcto** · 1 intento(s)

Escribe la línea que saca el último elemento de lista, como hace pop en una pila.

```
lista.remove(lista.size() - 1);
```

En NetBeans · **Correcto** · 1 intento(s)

Pila con ArrayList. En NetBeans, con import java.util.ArrayList;, escribe este código y agrega un while que saque e imprima todo usando pila.remove(pila.size() - 1) mientras la pila no esté vacía. Ejecuta y anota la salida.

```
c
b
a
```

Opción múltiple · **Correcto** · 1 intento(s)

En un ArrayList, ¿qué dice lista.size()?

```
Cuántos datos hay guardados
```

Predice la salida · **Correcto** · 1 intento(s)

¿Qué imprime? Son 2 líneas.

```
[5, 15]
[5]
```

Opción múltiple · **Correcto** · 1 intento(s)

Con ArrayList<Integer> lista = new ArrayList<>(); escribes lista.offer(5); ¿Qué pasa?

```
No compila: ArrayList no tiene offer
```

Opción múltiple · **Correcto** · 1 intento(s)

Usando ArrayList, ¿qué diferencia una pila de una cola?

```
El número dentro de remove: size() - 1 para pila, 0 para cola
```

Lista enlazada como pila y cola

Opción múltiple · **Correcto** · 1 intento(s)

En una pila con lista enlazada haces push(40). ¿Dónde queda el nodo nuevo?

```
Adelante: pasa a ser la cabeza
```

Opción múltiple · **Correcto** · 1 intento(s)

¿Qué significa que el siguiente del último nodo sea null?

```
Que no hay más nodos: es el final
```

Opción múltiple · **Correcto** · 1 intento(s)

En ColaEnlazada, ¿para qué sirve la referencia ultimo?

```
Para enganchar el nodo nuevo sin recorrer toda la lista
```

Predice la salida · **Correcto** · 1 intento(s)

¿Qué imprime? Son 2 líneas.

```
3
1
```

Escribe código · **Correcto** · 1 intento(s)

Escribe las tres líneas del método push de PilaEnlazada: crear el nodo, engancharlo a la cabeza y convertirlo en la cabeza.

```
Nodo nuevo = new Nodo (x);
nuevo.siguiente = cabeza;
cabeza = nuevo;
```

En NetBeans · **Correcto** · 3 intento(s)

Tu propia cola enlazada. En NetBeans crea las clases Nodo y ColaEnlazada como están arriba, escribiéndolas tú. En tu main escribe este código, ejecuta y anota toda la salida.

```
10
20
30
40
underflow: la cola esta vacia
-1
```

Opción múltiple · **Correcto** · 1 intento(s)

En push, si escribes cabeza = nuevo; antes de nuevo.siguiente = cabeza;, ¿qué pasa?

```
El nuevo nodo se apunta a sí mismo y se pierde el resto de la lista
```

¿Qué tan rápido es? La notación O

Opción múltiple · **Correcto** · 1 intento(s)

¿Cómo se lee O(n)?

```
"O de n"
```

Opción múltiple · **Correcto** · 1 intento(s)

Un ArrayList usado como cola tiene 1.000 datos. Al hacer remove(0), ¿cuántos datos se mueven?

```
999
```

Opción múltiple · **Correcto** · 2 intento(s)

¿Qué costo tiene poll() en una cola LinkedList?

```
O(1)
```

Opción múltiple · **Correcto** · 1 intento(s)

¿Qué costo tiene offer en una PriorityQueue?

```
O(log n)
```

Opción múltiple · **Correcto** · 1 intento(s)

¿Por qué get(i) es O(n) en LinkedList y O(1) en ArrayList?

```
LinkedList camina nodo por nodo; ArrayList va directo a la casilla
```

Interfaz vs. clase

Opción múltiple · **Correcto** · 1 intento(s)

¿Por qué new Queue<>() no compila?

```
Porque sus métodos no tienen cuerpo: no habría código que ejecutar
```

Opción múltiple · **Correcto** · 4 intento(s)

¿Cuál de estas líneas compila?

```
Queue<String> q = new LinkedList<>();
```

Opción múltiple · **Correcto** · 3 intento(s)

Una clase dice implements Queue pero le falta el método poll(). ¿Qué pasa?

```
No compila
```

Escribe código · **Correcto** · 4 intento(s)

Escribe la línea que crea una cola de enteros llamada cola, con la interfaz Queue como tipo y LinkedList como objeto.

```
   Queue<Integer> cola = new LinkedList<>();
```

Predice la salida · **Correcto** · 1 intento(s)

¿Qué imprime? Son 2 líneas.

```
2
5
```

Opción múltiple · **Correcto** · 2 intento(s)

En Queue<Integer> cola = new LinkedList<>(); ¿qué es el objeto?

```
El LinkedList que fabrica new
```

Opción múltiple · **Correcto** · 1 intento(s)

¿Cuándo revisa Java si un método existe en el tipo de la variable?

```
Al compilar, antes de correr el programa
```

Opción múltiple · **Correcto** · 1 intento(s)

class LinkedList implements Deque. ¿Qué recibe LinkedList de Deque?

```
Obligaciones: métodos que debe escribir
```

Opción múltiple · **Correcto** · 1 intento(s)

¿Compila Queue<Integer> q = new MiCola<>();?

```
Solo si tú escribiste la clase MiCola y cumple Queue
```

Opción múltiple · **Correcto** · 2 intento(s)

Queue<Integer> a = new LinkedList<>(); Queue<Integer> b = new LinkedList<>(); Queue<Integer> c = a; ¿Cuántos objetos hay?

```
2
```

El mapa de las colecciones

Opción múltiple · **Correcto** · 1 intento(s)

¿De qué clase hereda Stack?

```
Vector
```

Opción múltiple · **Correcto** · 1 intento(s)

¿Qué interfaz cumple en Java el papel de "interfaz de pila"?

```
Deque
```

Opción múltiple · **Correcto** · 2 intento(s)

¿Cuál de estas clases puede funcionar como pila y también como cola?

```
ArrayDeque
```

Opción múltiple · **Correcto** · 2 intento(s)

Queue hereda de Collection. Por eso una cola también tiene...

```
size() y contains()
```

Opción múltiple · **Correcto** · 3 intento(s)

¿De dónde le vienen push y pop a ArrayDeque?

```
De Deque
```

Opción múltiple · **Correcto** · 1 intento(s)

¿De dónde le vienen push y pop a Stack?

```
Son métodos propios de la clase Stack
```

Opción múltiple · **Correcto** · 1 intento(s)

¿Cuál de estas NO es una Collection?

```
HashMap
```

Opción múltiple · **Correcto** · 1 intento(s)

¿Por qué LinkedList tiene get(i)?

```
Porque implementa List
```

Opción múltiple · **Correcto** · 1 intento(s)

¿Cuál de estas es una interfaz?

```
Queue
```

La clase Stack

Predice la salida · **Respuesta vista** · 3 intento(s)

¿Qué imprime el ejemplo de la pila de números? Son 2 líneas.

```
1231.5678
1234.5678
```

Opción múltiple · **Correcto** · 1 intento(s)

¿Qué indica la L en pila.push(12L)?

```
Que es un long (entero largo)
```

Opción múltiple · **Correcto** · 1 intento(s)

¿Por qué pila.get(0) es un problema en un Stack?

```
Porque saca el elemento del fondo y rompe LIFO
```

Opción múltiple · **Correcto** · 1 intento(s)

¿Stack es otra estructura al mismo nivel que Vector?

```
No: Vector es una lista y Stack es una pila construida encima de ella
```

Predice la salida · **Correcto** · 2 intento(s)

En un Stack, ¿qué imprime? Son 2 líneas.

```
[1,2,3]
3
```

Excepciones y try-catch

Predice la salida · **Correcto** · 1 intento(s)

¿Qué imprime? La pila empieza vacía.

```
pila vacia
fin
```

Opción múltiple · **Correcto** · 1 intento(s)

En el ejercicio anterior, ¿por qué no se imprime "Saque algo"?

```
Porque la línea anterior lanzó la excepción y el resto del try se salta
```

Opción múltiple · **Correcto** · 1 intento(s)

Si no hubiera try-catch y haces pop en una pila vacía, ¿qué pasa?

```
El programa se detiene con EmptyStackException
```

Escribe código · **Correcto** · 2 intento(s)

Escribe un ciclo while que saque e imprima todos los elementos de la pila llamada pila, sin usar try-catch.

```
while (!pila.isEmpty()){
System.out.println(pila.pop());}
```

La cola y la interfaz Queue

Predice la salida · **Respuesta vista** · 3 intento(s)

¿Qué imprime el código "Una cola paso a paso"? Son 5 líneas.

```
[0,1,2,3,4]
0
[1,2,3,4}
 4
1
```

Opción múltiple · **Correcto** · 2 intento(s)

¿Qué hace poll() si la cola está vacía?

```
Devuelve null
```

Opción múltiple · **Correcto** · 2 intento(s)

¿Qué hace remove() si la cola está vacía?

```
Lanza NoSuchElementException
```

Opción múltiple · **Correcto** · 1 intento(s)

¿Para qué existe offer si ya existe add?

```
Para colas con cupo fijo: devuelve false en vez de lanzar una excepción
```

Predice la salida · **Correcto** · 1 intento(s)

¿Qué imprime? Son 2 líneas.

```
[10,30]
10
```

Opción múltiple · **Correcto** · 1 intento(s)

En una cola con datos, ¿qué diferencia hay entre remove() y poll()?

```
Ninguna: los dos sacan el frente. Solo cambian cuando la cola está vacía
```

Opción múltiple · **Correcto** · 1 intento(s)

¿Qué decide que una cola se comporte como FIFO?

```
La clase del objeto, por ejemplo ArrayDeque
```

Capacidad y LinkedList

Predice la salida · **Correcto** · 1 intento(s)

En el ejemplo del ascensor para 2, ¿qué imprimen los tres println?

```
true
true
false
```

Opción múltiple · **Correcto** · 1 intento(s)

new PriorityQueue<>(10) crea una cola...

```
Que empieza con espacio para 10 y crece si hace falta
```

Opción múltiple · **Correcto** · 2 intento(s)

Con Queue<String> fila = new LinkedList<>(); ¿compila fila.get(0)?

```
No, porque el tipo de la variable es Queue y Queue no tiene get
```

Escribe código · **Correcto** · 1 intento(s)

Escribe la línea que crea una cola de String llamada ascensor con capacidad fija de 3 personas.

```
Queue<String> ascensor = new ArrayBlockingQueue<>(3);
```

Deque y ArrayDeque

Predice la salida · **Correcto** · 1 intento(s)

¿Qué imprime? Son 2 líneas.

```
[3,2,1]
```

```
3
```

Escribe código · **Correcto** · 4 intento(s)

Escribe la línea que crea una pila de String llamada historial, con Deque como tipo y ArrayDeque como objeto.

```
Deque<String> historial = new ArrayDeque<>();
```

En NetBeans · **Respuesta vista** · 3 intento(s)

El botón Atrás. Crea en NetBeans la clase EjercicioPila con su main e import java.util.Stack;. Escribe este código y, al final, agrega el ciclo while con isEmpty() que saque e imprima todo lo que quede. Ejecuta y escribe aquí toda la salida de la consola.

```
Deportes
noticias
noticias
inicio
```

Predice la salida · **Correcto** · 1 intento(s)

¿Qué imprime? Son 3 líneas.

```
[3,1,2]
3
1
```

Opción múltiple · **Correcto** · 1 intento(s)

Queue<Integer> c = new LinkedList<>(); y escribes c.push(5); ¿Qué pasa?

```
No compila: push no está en Queue
```

Opción múltiple · **Correcto** · 3 intento(s)

¿Con qué tipo declaras un ArrayDeque para usarlo como pila?

```
Deque
```

Escribe código · **Correcto** · 2 intento(s)

Escribe la línea que crea una cola de String llamada turnos, con Queue como tipo y ArrayDeque como objeto.

```
 Queue<String> turnos = new ArrayDeque<>();
```

Opción múltiple · **Correcto** · 1 intento(s)

Con Deque<Integer> pila = new ArrayDeque<>(); ¿la regla LIFO queda totalmente protegida?

```
No: no hay get(i), pero se pueden usar los dos extremos. Hay que usar solo push, pop y peek
```

¿Toda cola es FIFO?

Opción múltiple · **Correcto** · 1 intento(s)

¿Cuál de estas afirmaciones es verdadera?

```
Queue define las operaciones; la clase decide el orden
```

Predice la salida · **Correcto** · 1 intento(s)

¿Qué imprime el ejemplo de asLifoQueue?

```
3
```

En NetBeans · **Correcto** · 1 intento(s)

Turnos del banco. En NetBeans, ejecuta este código con LinkedList y anota la salida. Luego cambia new LinkedList<>() por new PriorityQueue<>(), ejecuta otra vez y anota la nueva salida debajo. Escribe las 6 líneas en orden.

```
Sara
Ana
Luis
Ana
Luis
Sara
```

Colas de prioridad

Predice la salida · **Correcto** · 1 intento(s)

¿Qué imprime el código de "La trampa de imprimirla"? Son 3 líneas.

```
1
[1,3,8,5]
1 3 5 8
```

Opción múltiple · **Correcto** · 1 intento(s)

En new PriorityQueue<String>(), ¿qué decide el orden?

```
Comparable de String: orden alfabético
```

Opción múltiple · **Correcto** · 1 intento(s)

¿Qué hace Comparator.reverseOrder() en una PriorityQueue de números?

```
Hace que salga el mayor primero
```

Opción múltiple · **Correcto** · 1 intento(s)

¿Por qué System.out.println(pq) no muestra la cola ordenada?

```
Por dentro es un montículo: solo el frente está garantizado
```

Escribe código · **Correcto** · 2 intento(s)

Escribe la línea que crea una cola de prioridad de enteros llamada pq donde salga primero el mayor.

```
PriorityQueue<Integer> pq = new PriorityQueue<>(Comparator.reverseOrder());
```

En NetBeans · **Correcto** · 2 intento(s)

Palabras cortas primero. En NetBeans, escribe este código y agrega un while que saque e imprima con poll. Ejecuta y anota la salida. Luego quita el Comparator (deja new PriorityQueue<>()), ejecuta otra vez y anota la nueva salida debajo. Escribe las 6 líneas en orden.

```
uva
pera
manzana
manzana
pera
uva
```

Opción múltiple · **Correcto** · 1 intento(s)

En el arreglo del montículo, ¿en qué posición está el hijo izquierdo de la posición i?

```
2 * i + 1
```

Opción múltiple · **Correcto** · 1 intento(s)

¿Cuál es el padre de la posición 4?

```
1
```

Predice la salida · **Correcto** · 1 intento(s)

¿Qué imprime? Son 3 líneas. Usa el simulador si lo necesitas.

```
[1,2,7,4]
1
[2,4,7]
```

Opción múltiple · **Correcto** · 1 intento(s)

¿Qué hace sift-up al hacer offer?

```
El nuevo sube intercambiándose con su padre mientras sea menor
```

Opción múltiple · **Correcto** · 1 intento(s)

¿Por qué offer y poll en PriorityQueue cuestan O(log n)?

```
Porque solo se reacomoda un camino del árbol, de altura log n
```

Repaso: ¿verdadero o falso?

Opción múltiple · **Correcto** · 2 intento(s)

Que la pila quede vacía después de un pop es un underflow.

```
Falso
```

Opción múltiple · **Correcto** · 2 intento(s)

Stack no se recomienda porque, al heredar de Vector, deja tocar el medio.

```
Verdadero
```

Opción múltiple · **Correcto** · 2 intento(s)

En Java existe una interfaz llamada Pila.

```
Falso
```

Opción múltiple · **Correcto** · 2 intento(s)

Una interfaz trae el código de sus métodos.

Falso

Opción múltiple · **Correcto** · 1 intento(s)

Con una variable de tipo interfaz se usa el punto, pero solo con los métodos que la interfaz menciona.

Verdadero

Opción múltiple · **Correcto** · 2 intento(s)

ArrayList tiene los métodos offer y poll.

Falso

Opción múltiple · **Correcto** · 2 intento(s)

En un ArrayList<Integer>, lista.remove(10) borra el número 10.

Falso

Opción múltiple · **Correcto** · 1 intento(s)

pop en una pila con arreglo no borra el dato: solo baja tope.

Verdadero

Opción múltiple · **Correcto** · 2 intento(s)

Imprimir una PriorityQueue la muestra ordenada.

Falso

Opción múltiple · **Correcto** · 1 intento(s)

Toda cola de Java es FIFO.

Falso

Opción múltiple · **Correcto** · 2 intento(s)

HashMap es una Collection.

Falso

Opción múltiple · **Correcto** · 2 intento(s)

Un arreglo int[] tiene el método size().

Falso

Opción múltiple · **Correcto** · 1 intento(s)

En un ArrayDeque, push y add meten por el mismo extremo.

Falso

Opción múltiple · **Correcto** · 2 intento(s)

Declarar una cola como Queue impide del todo romper FIFO.

Falso

Opción múltiple · **Correcto** · 1 intento(s)

La misma línea cola.offer(5) puede ejecutar códigos distintos según el objeto.

Verdadero

Opción múltiple · **Correcto** · 2 intento(s)

new PriorityQueue<>(10) guarda como máximo 10 elementos.

Falso

Reto: usar pilas y colas de verdad

Opción múltiple · **Correcto** · 1 intento(s)

Para verificar si los paréntesis están bien cerrados, ¿qué estructura se usa y por qué?

Una pila: el último paréntesis que se abre es el primero que se cierra

Predice la salida · **Correcto** · 1 intento(s)

Con el método balanceado de arriba, ¿qué imprime? Son 3 líneas.

```
true
false
false
```

Escribe código · **Respuesta vista** · 3 intento(s)

Sin mirar arriba, escribe lo que va dentro del for: si c es (, apílalo; si es) y la pila está vacía, devuelve false; si no, desapila.

```
if (c == ´(´) {
pila.push (c);
} else if ( c == ´)´) {
if (pila.isEmpty()) {
return false;
}
pila.pop();
}
```

En NetBeans · **Correcto** · 1 intento(s)

Paréntesis en NetBeans. Escribe el método balanceado dentro de tu clase con main (con import java.util.*;) y prueba estos cuatro textos. Anota la salida.

```
true
false
false
true
```

En NetBeans · **Correcto** · 1 intento(s)

Turnos con urgencias. Ejecuta este código en NetBeans y anota la salida.

```
urgencia 1
urgencia 2
urgencia 3
turno Ana
Turno Luis
```

En NetBeans · **Correcto** · 1 intento(s)

Deshacer. Escribe y ejecuta en NetBeans. Anota la salida.

```
Deshacer: color rojo
Deshacer: negrita
Queda: escribir Hola
```

¿Cuál usarías?

Opción múltiple · **Correcto** · 3 intento(s)

Deshacer los últimos cambios de un formulario.

`Pila`

Opción múltiple · **Correcto** · 1 intento(s)

Atender solicitudes en el orden en que llegaron.

`Cola`

Opción múltiple · **Correcto** · 1 intento(s)

Despachar primero los pedidos marcados como urgentes.

`Cola de prioridad`

Opción múltiple · **Correcto** · 2 intento(s)

Volver a la pantalla anterior de una aplicación.

`Pila`